Configuración del Entorno de Desarrollo y Primer Programa en C++

Estudiante: Ian Quiroga **Curso:** 5.º año — 3.ª División (A-B) **Materia:** Laboratorio de Programación (LPR) **Institución:** E.E.S.T. N.º 1 «Eduardo Ader» — Vicente López **Docente:** Mansilla Muñoz, York Elías **Actividad:** 2 — El Taller del Programador **Fecha:** 2026

Introducción

Esta actividad tiene un objetivo doble. Por un lado, configurar un entorno de desarrollo completo para poder escribir y ejecutar programas en C++ de forma autónoma. Por otro lado, comprobar en la práctica que ese entorno funciona compilando y ejecutando el primer programa de la línea: el clásico *Hola Mundo*.

La pregunta que guía el desarrollo es simple: **¿qué herramientas necesito y qué conceptos debo dominar para pasar de un archivo de texto a un programa que se ejecuta?** La respuesta corta es que hacen falta tres piezas —editor, compilador y depurador— y que por debajo de todas ellas hay una sola cosa que debe estar bien configurada: el `Path`.

Desarrollo

Las herramientas del laboratorio

En el desarrollo de software se elige la herramienta según la tarea. Las recomendadas para este nivel son:

1. **Visual Studio Code.** Editor de código ligero, potente y personalizable

mediante extensiones. Es el estándar actual de la industria porque no obliga a instalar un entorno pesado para empezar a trabajar.

2. **Visual Studio Community.** IDE completo y robusto, orientado a proyectos

grandes de nivel empresarial y aplicaciones de escritorio complejas.

3. **Sublime Text.** Editor de texto sofisticado y extremadamente rápido para

ediciones veloces de archivos sueltos.

Se optó por **VS Code** por ser el equilibro adecuado entre potencia y ligereza, y por su integración directa con el compilador de la terminal integrada.

La cadena de transformación: del código fuente al ejecutable

El glosario de la actividad define cinco conceptos que en realidad forman una cadena. Entender esa cadena es el objetivo académico de esta parte:

```
Código fuente  ■■[Compilador]■■■  Código objeto  ■■[Enlazador]■■■  Ejecutable
   (.cpp)                                                            (.exe)
```

• **Código fuente**: el conjunto de líneas de texto que escribimos en un

lenguaje. Es lo que nosotros entendemos, pero la computadora no.

• **Compilador**: el traductor de alto nivel. Toma el código fuente y lo

transforma en código de máquina (binario) o de bajo nivel, creando un archivo ejecutable.

• **Depurador (Debugger)**: el detector de errores. Permite ejecutar el programa

paso a paso para encontrar errores de lógica o *bugs* que no se ven a simple vista.

• **IDE**: el entorno que junta editor, compilador y depurador en una sola

ventana.

• **Terminal o consola**: ventana de texto plano donde se escriben comandos

directos al sistema operativo, sin usar el mouse.

• **Path (ruta de sistema)**: la lista de carpetas donde el sistema operativo

busca los ejecutables. **Si el compilador no está configurado en el `Path`, la terminal no sabrá dónde encontrar el comando `g++`.**

Ese último punto explica por qué la instalación de MSYS2/MinGW no alcanza por sí sola: si `g++` no está en el `Path`, la terminal responde que no reconoce el comando, y el error parece del programa cuando en realidad es de configuración.

Configuración de VS Code para C/C++

Para que el editor sea realmente útil se instalaron tres extensiones (panel `Ctrl + Shift + X`):

1. **C/C++ (Microsoft)**: fundamental para que el editor entienda la sintaxis,

completa las llaves y marca los errores en tiempo real.

2. **C/C++ Extension Pack**: agrega herramientas de depuración y navegación de

código.

3. **Code Runner** (opcional pero recomendado): permite correr el código con un

clic, sin escribir el comando de compilación a mano.

Sin el compilador MinGW (GCC), VS Code es solamente un bloc de notas con colores.

El programa de prueba

El archivo `src/main.cpp` sigue el esqueleto provisto por el docente:

```
#include <iostream>

using namespace std;

int main()
{
```

```
    cout << "==================================================" << endl;
    cout << "   Hola Mundo! Bienvenido al Taller de LPR 2026.   " << endl;
    cout << "   Estudiante: Ian Quiroga   " << endl;
    cout << "==================================================" << endl;

    return 0;
}
```

Dos decisiones del código merecen explicación:

• `#include <iostream>` importa la biblioteca estándar de entrada y salida,

que es la que provee `cout` y `cin`. Sin esa línea, el compilador no reconoce esos nombres.

• `using namespace std;` evita anteponer `std::` antes de cada comando de

consola. Sin él, habría que escribir `std::cout`.

• `return 0;` le indica al sistema operativo Windows que el programa

finalizó con éxito. Es el código de salida que el sistema usa para saber si la ejecución fue correcta.

Se respetó la regla obligatoria de modificar la salida agregando el nombre y apellido reales del estudiante antes de compilar y entregar.

Compilación y ejecución desde PowerShell

Desde la terminal integrada de VS Code (`Ctrl + Ñ`), parado en la raíz de la carpeta `HOLAMUNDO`:

```
g++ src/main.cpp -o src/holamundo.exe
.\src\holamundo.exe
```

El flag `-o src/holamundo.exe` le indica al compilador el nombre y la ubicacion del ejecutable generado, en lugar del nombre por defecto `a.exe`.

Conclusión

La actividad puede parecer trivial —un `Hola Mundo` de veinte líneas— pero el trabajo real no está en el código: está en el entorno. Saber por qué `g++` no se reconoce, dónde está el `Path` y qué hace exactamente un compilador evita perder horas frente a errores que no son del programa.

Las tres conclusiones principales son: la elección del IDE depende del tipo y del tamaño del proyecto, no de la moda; el `Path` es el punto de fallo más frecuente al configurar un entorno nuevo; y la diferencia entre un editor de texto y un IDE es que el IDE además de escribir código, lo compila y lo depura.

La documentación técnica de un entorno —qué se instaló y con qué comandos se verificó— es tan parte del trabajo como el código mismo, y es exactamente lo que pide un Técnico en Programación cuando se incorpora a un equipo.

Referencias

• Microsoft. (2024). *Visual Studio Code: Documentation*. Microsoft Learn.

<https://code.visualstudio.com/docs>

• Microsoft. (2024). *C/C++ for Visual Studio Code Extension*. Marketplace.
<https://marketplace.visualstudio.com/items?itemName=ms-vscode.cpptools>

• Breland, A. (2023). *MSYS2 — Installing tools for the MSYS2 environment*.
MSYS2. <https://www.msys2.org/>

• GCC. (2024). *GCC 13.2.0 documentation*. Free Software Foundation.
<https://gcc.gnu.org/onlinedocs/gcc-13.2.0/gcc/>

• Microsoft. (2024). *Path (Environment Variable)*. Microsoft Learn.
<https://learn.microsoft.com/es-es/windows-server/administration/windows-commands/path>